RB 협동로봇 외부 스크립트 인터페이스 정의서 (간이)

- 대상: Rainbow Robotics 협동로봇 제어기, 외부 스크립트 방식 (TCP)
- 근거: 공식 문서 외부 스크립트 API, UI Script 와 이 저장소 도구(`rb_servo_gui`, `jb2_gui`)의 구현·실험
- 표기: **[문서]** 공식 문서에 있는 내용, **[관측]** 실험에서 확인했지만 문서에는 없는 내용 (제어기·펌웨어에 따라 다를 수 있음)
- 예시 값(IP 10.0.2.7, 관절값)은 이 프로젝트의 환경이며, **모션 명령은 반드시 시뮬레이션 모드에서 먼저 시험**

1. 연결

포트	용도	방향	형식
5000	명령 포트	외부 → 제어기 (명령), 제어기 → 외부 (응답)	텍스트 한 줄 = 명령 하나
5001	데이터 포트	`reqdata` 요청 → 이진 상태 패킷 응답	580바이트 고정 길이 [문서: 포트 역할], [관측: 길이·오프셋]

- 두 포트는 **별도 TCP 연결**. 명령 전송과 상태 읽기를 서로 다른 연결(스레드)에서 동시에 해도 됨 [관측]
- 명령은 **받는 즉시 실행**되며 별도 확인 절차가 없음. 모드와 목표값을 보내기 전에 확인
- 제어기 IP 는 제어기 화면(LCD)에 표시된 값

2. 명령 포트 (5000)

2.1 전송 규칙

- 한 줄에 명령 하나, 끝에 개행(`\n`)
- 응답 [문서]: 유효한 명령이면 `The command was executed` , 허용되지 않는 명령이면 `The command is not allowed`
- **여러 줄을 연속 전송할 때**
  - 응답을 기다리지 않고 몰아 보내면 TCP 가 줄을 합치거나 쪼개 구문 오류가 남 [관측]. 증상: 태블릿에 `syntax error at ' ' in this line: move_jb2_add(j`
  - 대응: Nagle 끄기(`TCP_NODELAY`) + 한 줄마다 응답을 받은 뒤 다음 줄 전송 (특히 `move_jb2_add` 반복 구간)
- 관절값: 임시 변수 `jnt[j0, j1, j2, j3, j4, j5]` (deg, 별도 선언 불필요) [문서]

2.2 모드 전환

명령	설명
`pgmode simulation`	시뮬레이션 모드: 로봇은 움직이지 않고 지령값(`jnt_ref`)만 갱신
`pgmode real`	실기 모드: 실제 구동

- 시뮬레이션에서 엔코더값 `jnt_ang` 는 갱신되지 않고 0 으로 고정 [관측] → 시뮬레이션은 `jnt_ref` , 실기는 `jnt_ang` 로 위치를 읽음

2.3 모션 명령

명령	인자	설명
`move_j(J, v, a [, wait])`	J: 관절 목표(deg), v: 속도(deg/s), a: 가속도(deg/s²), wait: 0 대기 안 함 / 1 완료까지 대기 (기본 1)	단일 관절 이동
`move_jb2_clear()`	없음	jb2 점 리스트 초기화
`move_jb2_add(J, v, a, 0, blend)`	4번째 인자는 더미(0), blend: 블렌드율(%)	jb2 리스트에 점 추가
`move_jb2_run([wait])`	wait: 0 이면 시작만 하고 반환, 1 대기 (기본 1)	리스트의 점을 이어서 실행
`move_servo_j(J, t1, t2, gain, alpha)`	아래 표	실시간 서보: 목표 관절각을 계속 갱신

`move_servo_j` 인자 [문서]

인자	의미	범위
J	목표 관절각 (deg)	-360 ~ 360
t1	도달 시간 (s)	≥ 0.002
t2	유지 시간 (s)	0.02 < t2 < 0.2
gain	속도 추종 비율	> 0
alpha	저역 통과 필터	0 < alpha < 1

2.4 작업(task) 제어

명령	설명
`task load <파일명>`	작업 파일(.wsl) 불러오기
`task repeat <횟수>`	반복 횟수
`task play once`	불러온 작업 실행
`task pause` / `task resume_a`	일시 정지 / 재개

명령	설명
`task stop`	작업 정지 요청 (**비상정지가 아님**)

(`resume_b` 도 문서에 있음)

3. 명령 예시

3.1 시퀀스 (한 줄씩, 시뮬레이션 모드)

```
pgmode simulation
move_j(jnt[66.757,-42.499,49.902,-6.537,113.220,0.363], 50, 120)
move_jb2_clear()
move_jb2_add(jnt[66.9632,-42.2424,51.4177,-9.6759,112.9050,0.2800], 50, 120, 0, 100)
move_jb2_add(jnt[67.0438,-41.5110,53.5442,-14.9398,111.9343,0.1912], 50, 120, 0, 100)
move_jb2_run(0)
```

```
pgmode simulation
move_servo_j(jnt[66.9632,-42.2424,51.4177,-9.6759,112.9050,0.2800], 0.02, 0.04, 1, 1)
move_servo_j(jnt[67.0438,-41.5110,53.5442,-14.9398,111.9343,0.1912], 0.02, 0.04, 1, 1)
```

- servo 는 **주기적으로 계속** 보내야 함 (예: 20 ms 마다 다음 목표, `t1` = 주기). 한꺼번에 보내면 앞 목표가 덮어써질 것으로 보임 [추정, 직접 시험하지 않음]

3.2 터미널(nc)

```
nc 10.0.2.7 5000                          # 명령 포트에 접속 후 한 줄씩 입력
pgmode simulation
move_j(jnt[0,0,0,0,0,0], 20, 40)
```

3.3 Python (명령 + 상태 읽기)

```
import socket, struct

IP = "10.0.2.7"
cmd = socket.create_connection((IP, 5000)); cmd.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, 1)
dat = socket.create_connection((IP, 5001))

def send(line):                       # 한 줄 전송 (개행 포함)
    cmd.sendall((line.strip() + "\n").encode())

def read_state():                     # reqdata → 580바이트
    dat.sendall(b"reqdata\n")
    buf = b""
    while len(buf) < 580:
        buf += dat.recv(580 - len(buf))
    assert buf[0] == 0x24 and buf[3] == 0x03, "헤더 불일치"
    return dict(jnt_ref=struct.unpack_from("<6f", buf, 8), jnt_ang=struct.unpack_from("<6f", buf, 32),
                robot_state=struct.unpack_from("<i", buf, 340)[0], sim=struct.unpack_from("<i", buf, 404)[0])

send("pgmode simulation")
print(read_state())
```

4. 데이터 포트 (5001): `reqdata`

- 요청: 텍스트 `reqdata\n` → 응답: 580바이트 이진, **리틀엔디안** [관측]
- 헤더 확인: 첫 바이트 `0x24`, 네 번째 바이트 `0x03`

이 프로젝트에서 읽는 필드 (오프셋은 바이트) [관측]

오프셋	형식	필드	설명
4	float	제어기 시간	
8	float × 6	`jnt_ref`	관절 **지령**값 (deg). 시뮬레이션의 위치
32	float × 6	`jnt_ang`	관절 **엔코더**값 (deg). 실기의 위치, 시뮬레이션에서는 0
104	float × 6	`tcp_pos`	TCP 위치 (이 프로젝트에서는 사용하지 않음)
340	int	`robot_state`	1 = 정지(idle), 3 = 실행 중
404	int	`real_vs_simulation_mode`	1 = 시뮬레이션, 0 = 실기

- 위 외의 필드(속도 배율 등)는 공식 문서의 "Request 데이터 구조 매뉴얼" 참조
- 모드를 확인하는 습관: 모션 명령 전후에 `real_vs_simulation_mode` 를 읽어 의도한 모드인지 검사

5. 사용상 주의

1. **안전**: 명령은 확인 없이 즉시 실행. `pgmode real` 상태의 모션은 실제 로봇이 움직임. `task stop` 은 비상정지가 아니므로 물리 E-Stop 을 손이 닿는 곳에 둘 것
2. **완료 확인**: 모션 완료는 측정각이 목표에 도달 + `robot_state` = 1 로 판단. 시뮬레이션에서 `jnt_ang`(0)로 판단하면 도달하지 않았는데 도달로 오인함
3. **`move_servo_j` 범위**: `t2` 와 `alpha` 는 문서상 열린 구간. 경계값(`t2` = 0.2, `alpha` = 1.0)은 제어기가 거부하지 않았지만 문서 범위 밖일 수 있음
4. **servo 종료 시 오버슈트** [관측]: 마지막 지령 후 로봇이 (마지막 지령 속도 × `t2`) 만큼 더 나아가 멈춤. 마지막 목표를 여러 번 반복해 보내 속도를 0 으로 만들면 해소
5. **`move_jb2_*` 속도** [관측, 문서에 없음]
   - 설정 속도는 상한. 점이 촘촘하면(분할 수가 많으면) 도달 가능한 속도가 `√(accel × 구간 길이)` 정도로 낮아짐
   - 관절마다 속도·가속도 배율이 달라 어떤 관절이 포함되었는지에 따라 같은 설정에서도 속도가 달라짐 (예: J1·J2 단독은 설정의 약 0.68배, J3 약 0.93배, J4~J6 약 1.0배). 조합 규칙은 확정하지 못함
   - blend 가 클수록 경유점을 지나지 않고 코너를 깎음 (blend 0 은 점마다 정지하며 정확히 통과)
6. **명령 간격**: `move_servo_j` 의 `t1` 보다 짧은 간격으로 명령이 겹칠 때의 동작은 문서에 없음. `t1` 을 전송 주기와 같게 두는 방식으로 사용했음
7. 연속 전송 시 응답 대기(2.1절), 관절값 범위(-360 ~ 360)와 충돌 여부는 제어기가 검증해 주지 않으므로 호출 측에서 확인

6. 변경 이력

날짜	내용
2026-10-19	최초 작성 (간이판)